

0.33.3. La fonction `carre()` — tourner en carré

`#include <carre>` : `carre(x, y, tuile, taille, sens, vitesse, tours)` fait tourner la tuile en carré autour de (x, y).

1. Ce qu'il faut comprendre

Ce qu'elle fait : `carre(x, y, tuile, taille, sens, vitesse, tours)` fait faire à la tuile des **tours en carré** autour de la case (x, y). Taille 1 : un carré de 3 × 3 ; taille 2 : 5 × 5. Elle part du centre et y revient.

La ligne à écrire : `#include <carre>`.

Ses arguments : le centre, la tuile, la taille, le sens (1 : aiguilles d'une montre, -1 : l'autre), la vitesse en ms par pas (en clair), le nombre de tours.

Ce qu'elle coûte : environ **500 octets**, plus la police entière.

Essaie : `carre(10, 8, ALPHABET[0], 3, -1, 100, 2);`.

On retrouve `carre` dans 63 leçons

[0.34](#), [0.34.1](#), [0.34.2](#), [0.35](#), [0.35.1](#), [0.35.2](#), [0.36](#), [0.36.1](#), [0.36.2](#), [0.37](#), [0.37.1](#), [0.37.2](#), [0.38](#), [0.38.1](#), [0.38.2](#), [0.39](#), [0.39.1](#), [0.39.2](#), [0.40](#), [0.40.1](#), [0.40.2](#), [0.41](#), [0.41.1](#), [0.41.2](#), [0.42](#), [0.42.1](#), [0.42.2](#), [0.43](#), [0.43.1](#), [0.43.2](#), [0.44](#), [0.44.1](#), [0.44.2](#), [0.45](#), [0.45.1](#), [0.45.2](#), [0.46](#), [0.46.1](#), [0.46.2](#), [0.47](#), [0.47.1](#), [0.47.2](#), [0.48](#), [0.48.1](#), [0.48.2](#), [0.49](#), [0.49.1](#), [0.49.2](#), [0.50](#), [0.50.1](#), [0.50.2](#), [0.51](#), [0.51.1](#), [0.51.2](#), [0.52](#), [0.52.1](#), [0.52.2](#), [0.53](#), [0.53.1](#), [0.53.2](#), [0.54](#), [0.54.1](#), [0.54.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1349 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
carre()	440 o	← main()	– une fonction fournie par le langage
main()	38 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases	– attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer()	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer()	– efface des cases
CopierTuiles	18 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles	– la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 4	52 o sur place, dans carre() + AdresseCase (10 o)
effacer()	× 3	45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image()	× 4	12 o sur place, dans carre(), main() + AttendreImage (28 o)

Tes fonctions appelées :
carre() x 1 31 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

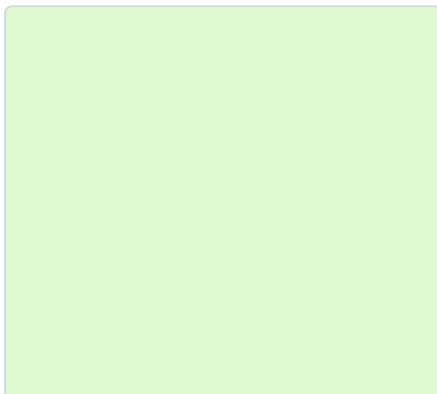
```
// ---- #include <carre> : tourner en carré ----  
  
// Un C qui tourne autour de la case (15, 4), en haut à droite.  
// (Le A au milieu, ce sera la leçon suivante.)  
  
#include <carre>      // carre() : des tours en carré  
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

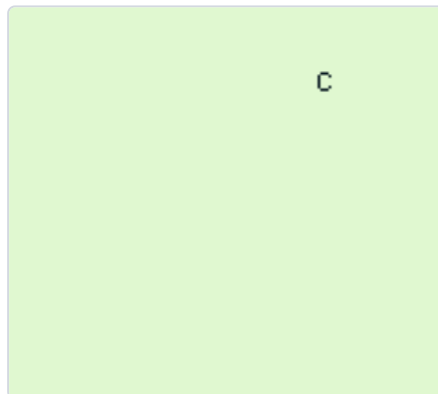
```
int main() {  
    carre(15, 4, ALPHABET[2], 1, 1, 250, 1);  // centre (15, 4), taille 1, un tour  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

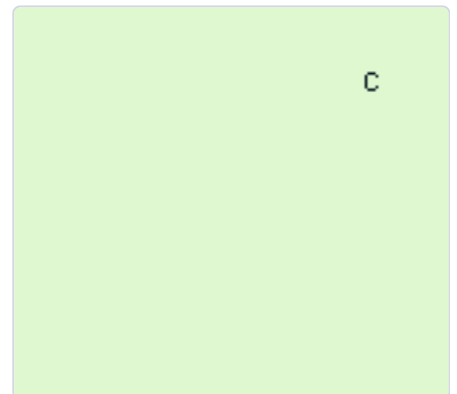
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



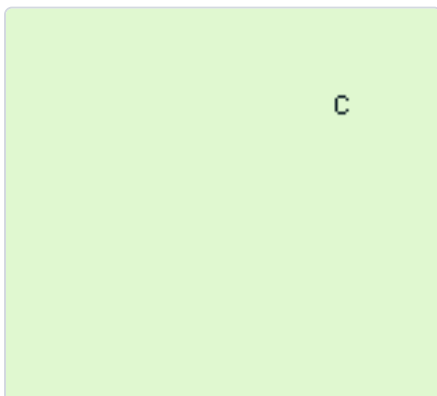
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un C qui fait un tour en carré autour de la case (15, 4), puis s'y arrête.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les quatre coins, dans le sens des aiguilles (14,3 16,3 16,5 14,5)

✓ le C revient au centre

Ensuite : **0.34. Un carré autour d'une lettre, avec carre** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++